OOP(Object Oriented Programming)

OOP lets you structure your code using classes and objects

```
def __init__(self,name,ip):
self.name=name
self.ip=ip
def status(self)
print(f"{self.name} at {self.ip} is running")

web_server= Server("WebServer","192.168.1.10")
web_server.status()
```

1. Class Server: itis a blue print of server
2. __init__: Runs when you create a new Server
3. self.name, self.ip: These are the server's details(Attributes)
4. status(): a function (Method) which will print the name and ip address
5. web_server: Server(...) creating an actual server (Object)
6. web_server.status: calling method of the class

TASK:1 Write a Program to call different methods of a class in Python -using class and Object

What is Inheritance?

- inheritance means one class can use the features of another class like a child and parent relationship
- a child can access all the methods of parent(inherits traits from parent)

```
#parent class
class Animal:
def __init__(self,name):
self.name=name
def speak(self):
print(f"{self.name} makes a sound")
#child classes
class Dog(Animal):
```

```
def speak(self):
print(f"{self.name} says Woof!")
class Cat(Animal):
def speak(self):
print(f"{self.name} says Meow")
class Tiger(Animal):
def speak(self):
print(f"{self.name} says Roar")

dog1= Dog("Buddy")
Cat1=Cat("Mimi")
tiger=Tiger("White Tiger")

dog1.speak()
Cat1.speak()
tiger.speak()
```

What is abstraction?

- Abstraction means showing only the essential features and hiding the complex details
- Like while pressing a start button of your car, you don't know the mechanism performed , but you only know that how to start

What is Method Overriding?:

when child overrides the method of a parent class then it is known as Method Overriding

```
from abc import ABC ,abstractmethod

class Car(ABC):
@abstractmethod
def start_engine(self):
pass
class Tesla(Car):
def start_engine(self):
print("Starting Tesla Engine Silently......!")
class Tata(Car):
def start_engine(self):
print("Starting TATA Engine Silently.......!")

my_car= Tesla()
my_car.start_engine()
```

```
my_car2=Tata()
my_car2.start_engine()
```

What is Encapsulation?

- wrapping of data (Encapsulating) to hide internal data
- Protecting Data from Direct Access
- Only Allowing access via Methods

```
class BankAccount:
def __init__(self,owner,balance):
self.owner=owner
self.__balance=balance #private variable
def deposit(self,amount):
self.__balance +=amount
def withdraw(self,amount):
if amount <=self.__balance:
self.__balance -=amount
else:
print("Insufficient Balance!")
def get_balance(self):
return self.__balance
#using the class
acc=BankAccount("Nikunj Soni",1000)
print("Loading Opening Balance:",acc.get_balance())

acc.deposit(200)
print("Balance After Deposit:",acc.get_balance())

acc.withdraw(100)
print("Balance After Withdraw:",acc.get_balance())
```